

Encontro 08

1. ECB com blocos A, B, A, C, B

Com a mesma chave, a saída em ECB fica **X, Y, X, Z, Y**.

Com isso, o observador consegue notar que os blocos 1 e 3 são iguais, que os blocos 2 e 5 também coincidem e que o bloco 4 é o único diferente. Ele também descobre o tamanho total da mensagem (5 blocos), mas não tem como saber o conteúdo de A, B e C nem descobrir a chave.

2. CBC com $E(x) = (x + 3) \bmod 16$, mensagem [4, 4] e IV = 1

i	Mi	valor anterior	XOR	Ci
1	4	1	$4 \oplus 1 = 5$	$5 + 3 = 8$
2	4	8	$4 \oplus 8 = C$	$C + 3 = F$

Resultado: C = [8, F].

Pra decifrar (desfazendo a cifra e depois o XOR):

- $M1 = (8 - 3) \oplus 1 = 5 \oplus 1 = 4$
- $M2 = (F - 3) \oplus 8 = C \oplus 8 = 4$

Chegamos de volta em [4, 4].

3. Por que o CBC cifra em sequência e decifra em paralelo

Na cifração em CBC, cada $C(i)$ depende do $C(i-1)$ que acabou de ser calculado, o que obriga o processo a ser estritamente sequencial.

Já na decifração, como $M(i) = D(C(i)) \oplus C(i-1)$ e todos os blocos cifrados já estão disponíveis logo no início, dá pra rodar as chamadas de D em paralelo sem problemas.

No modo CTR, tanto cifrar quanto decifrar podem rodar em paralelo porque cada bloco depende unicamente do seu próprio contador. Além disso, a decifração usa a função E (em vez de D), já que a cifra só serve pra gerar o keystream que depois é aplicado via XOR.

4. Formato T = N||j com N = 01

j	T em binário	T em hexadecimal
00	0100	4
01	0101	5

j	T em binário	T em hexadecimal
10	0110	6
11	0111	7

Assim, cabem no máximo 4 blocos. Se o sistema tentar enviar mais do que isso, precisa dar erro e interromper. Não dá pra deixar o contador estourar e voltar pra 00, porque isso reutilizaria os mesmos valores de contador.

5. $P = 20$, $Q = 30$ e fluxo $S = AA$

- $C = 20 \oplus AA = 8A$
- $C' = 30 \oplus AA = 9A$
- $C \oplus C' = 8A \oplus 9A = 10$, que é igual a $P \oplus Q$
- Sabendo P : $Q = P \oplus 10 = 20 \oplus 10 = 30$

Se o atacante não souber nenhum dos dois textos claros, ele só descobre a relação entre eles. O XOR não entrega o conteúdo individual, já que existem vários pares de valores que resultam em 10.

6. Mensagens de 31 e 32 bytes

Tamanho	AES-CBC com PKCS #7	CTR puro
31 bytes	$p = 16 - (31 \bmod 16) = 1$, total de 32 bytes	31 bytes
32 bytes	$p = 16$, total de 48 bytes	32 bytes

O CBC fica maior por conta do padding obrigatório, enquanto o CTR não precisa disso. Vale lembrar que a diferença de tamanho não altera em nada o nível de segurança.

7. CTR puro com $C = E7$ e $S = A6$, atacante troca C por $E6$

- Antes: $E7 \oplus A6 = 41$
- Depois: $E6 \oplus A6 = 40$

Alterando um bit no texto cifrado, o mesmo bit muda no texto claro recuperado (de 41 pra 40). Como o CTR funciona aplicando XOR diretamente com o keystream, qualquer alteração passa limpa pra mensagem final. O atacante não precisa saber a chave nem o fluxo, basta saber qual posição quer modificar.

8. AESGCM com 20 bytes e etiqueta de 16 bytes

- Saída da biblioteca: $20 + 16 = 36$ bytes
- Com o nonce de 12 bytes enviado à parte: $36 + 12 = 48$ bytes
- Se os dados associados estiverem errados: a verificação da etiqueta falha, a biblioteca lança InvalidTag e a mensagem não é liberada.

9. Corrigindo a frase

"Um nonce aleatório nunca se repete, um IV público é inseguro e a propagação de erros de CBC já autentica a mensagem."

Os três erros:

1. **"Nonce aleatório nunca se repete"** está errado. Ser aleatório não garante que não vai repetir, pode haver colisão. Um sistema real precisa de uma política de geração, limites de uso e troca de chave.
2. **"IV público é inseguro"** está errado. O IV pode ser público e ir junto do texto cifrado. No CBC o que importa é ele ser imprevisível, não secreto.
3. **"A propagação de erros do CBC já autentica"** está incorreto. O fato de um erro se espalhar não garante autenticidade nem detecta alterações propositais. Para garantir integridade e autenticidade, é preciso usar um MAC ou um modo AEAD como o AES-GCM.

10. Recepção com AES-GCM

O lado receptor pega a chave, o nonce, o texto cifrado, a tag e os dados associados. Tudo isso entra na função de decifração autenticada, que primeiro valida a tag.

- **Etiqueta válida:** a mensagem é liberada para a aplicação.
- **Etiqueta inválida:** a mensagem é rejeitada (InvalidTag) e o texto candidato não pode ser mostrado, interpretado nem executado.

Os caminhos são excludentes: se a tag não bater, o processo para ali e nada é processado.

Se uma mensagem antiga for capturada e reenviada sem alterações, a tag vai ser válida e ela passaria na verificação. Por isso, a aplicação precisa implementar proteção contra replay attacks, guardando estado (como uma tabela ou controle de sequência dos pacotes aceitos).

Desafio de fechamento

Se um sistema legado com AES-CBC retorna mensagens de erro distintas para "preenchimento inválido" e "senha incorreta", ele fica vulnerável a um ataque de padding oracle. O atacante pode modificar o ciphertext iterativamente e, observando os erros retornados, recuperar o texto claro bloco a bloco sem precisar da chave.

A solução correta é migrar para um modo autenticado (AEAD) como o AES-GCM, que valida a tag de autenticação antes de tentar decifrar ou checar o conteúdo. Padronizar as mensagens de erro ajuda, mas não resolve a causa raiz do problema.